Open in app ↗ Sign up Sign in

Medium Search Write

Modernizing the build process of existing Java EE applications with Maven

Dave Mulley Follow 7 min read · Apr 16, 2019

11

As part of modernizing existing traditional WebSphere applications to run on WebSphere Liberty and be deployed to IBM Cloud Private I often have to configure Maven to build the EAR file. While many new applications already use Maven or Gradle for the CI part of the CI/CD pipeline, many older applications are either built by hand or by using older technologies such as Ant.

Reorganize the project structure

I've settled on the following project structure when modernizing existing WebSphere applications (it is based on the example found here. There are folders for the Web Module (`war`), Java Module (`jar`) and the EAR file (`ear`). I've also added a `Liberty` folder that contains the deployment artifacts for Liberty (`server.xml` etc) which along with the `manifests`, `Dockerfile` and `Jenkinsfile` can be ignored at this point but are useful to plan for if your goal

is to automate build and deployment with Jenkins and deploy to a Kubernetes based runtime.

```
├── ear
│   └── pom.xml
├── jar
│   ├── src
│   │   └── main
│   │       ├── java
│   │       │   └── com
│   │       │       └── ...
│   │       │           └── file.java
│   │       ├── resources
│   │       │   └── com
│   │       │       └── ...
│   │       │           └── file.properties
│   └── pom.xml
├── war
│   ├── src
│   │   └── main
│   │       ├── java
│   │       │   └── com
│   │       │       └── ...
│   │       │           └── file.java
│   │       ├── resources
│   │       │   └── com
│   │       │       └── ...
│   │       │           └── file.properties
│   │       └── webapp
│   │           ├── images
│   │           ├── ...
│   │           ├── META-INF
│   │           └── WEB-INF
│   │               ├── ibm-web-bnd.xml
│   │               ├── ibm-web-ext.xml
│   │               └── web.xml
│   └── pom.xml
├── mainfests
│   └── deploy.yaml
├── liberty
│   ├── resources
│   │   └── library.jar
│   ├── bootstrap.properties
│   ├── jvm.options
│   ├── server.xml
│   └── server.env
├── Dockerfile
├── Jenkinsfile
└── pom.xml
```

Use the following steps to copy your source code in to the project.

1. Create the `war` folder as shown above.
2. Copy your source code in to the `war/src/java` folder (`.java` files only)
3. If you have any files other than .java in your source code that are currently packaged in the `war/WebContent/WEB-INF/classes` folder (often `.properties` files are packaged here) you'll need to put them in the `/war/src/resources` folder and they will be copied in to the correct location during the build process.
4. Copy the contents of your current `war/WebContent` folder apart from the `WEB-INF/classes` folder to the `/war/src/webapp` folder. This often includes `images` and `jsp` files as well as the `WEB-INF/web.xml` file and any IBM descriptor files.
5. **If** your application has a `Java Module`, create the `jar` folder as shown above and copy your source code to the `jar/src/java` folder (`.java` files only). Again, if you have non-java files that you want to be copied to the generated `.jar` file, add them to the `resources` folder.

Create the Root POM file

Create a pom.xml file in the root of your project using the template below.

```
<?xml version='1.0' encoding='utf-8'?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation=
 "http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

<parent>
 <groupId>net.wasdev.wlp.maven.parent</groupId>
 <artifactId>liberty-maven-app-parent</artifactId>
 <version>RELEASE</version>
</parent>
```

```
<!-- tag::packaging[] -->
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.ibm.issw</groupId>
 <artifactId>demoapp</artifactId>
 <version>0.0.1-SNAPSHOT</version>
 <packaging>pom</packaging>
<!-- end::packaging[] -->

<!-- tag::modules[] -->
 <modules>
 <module>jar</module>
 <module>war</module>
 <module>ear</module>
 </modules>
<!-- end::modules[] -->

<properties>
 <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
 <project.reporting.outputEncoding>UTF-
8</project.reporting.outputEncoding>
 <maven.compiler.source>1.8</maven.compiler.source>
 <maven.compiler.target>1.8</maven.compiler.target>
 <app.name>LibertyProject</app.name>
 <testServerHttpPort>9080</testServerHttpPort>
 <testServerHttpsPort>9443</testServerHttpsPort>

<package.file>${project.build.directory}/${app.name}.zip</package.fil
e>
 <packaging.type>usr</packaging.type>
 </properties>
</project>
```

You will want to change the following values:

1. In the `packaging` section change the `groupId` and `artifactId` to reflect the application that you are building. Note that these values are used in the other `pom.xml` files that we will create later.

2. Update the `modules` section to reflect the modules included in your application. If you don't have a `Java Module` remove the `<module>jar</module>` line.

Create the EAR POM file

Create a pom.xml file in the `/ear` folder of your project using the template below.

```
<?xml version='1.0' encoding='utf-8'?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation=
 "http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

<!-- tag::packaging[] -->
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.ibm.issw</groupId>
 <artifactId>demoapp.ear</artifactId>
 <version>0.0.1-SNAPSHOT</version>
 <packaging>ear</packaging>
<!-- end::packaging[] -->

<!-- tag::parent[] -->
 <parent>
   <groupId>com.ibm.issw</groupId>
   <artifactId>demoapp</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <relativePath>../pom.xml</relativePath>
 </parent>
<!-- end::parent[] -->

<dependencies>
 <dependency>
   <groupId>com.ibm.issw</groupId>
   <artifactId>demoapp.jar</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <type>jar</type>
 </dependency>
 <dependency>
   <groupId>com.ibm.issw</groupId>
   <artifactId>demoapp.war</artifactId>
    <version>0.0.1-SNAPSHOT</version>
  <type>war</type>
 </dependency>
 <!-- end::dependencies[] -->
</dependencies>
```

```
  <build>
   <plugins>
     <!-- tag::build[] -->
     <plugin>
       <groupId>org.apache.maven.plugins</groupId>
       <artifactId>maven-ear-plugin</artifactId>
       <version>3.0.1</version>
       <configuration>
        <modules>
         <jarModule>
           <groupId>com.ibm.issw</groupId>
           <artifactId>demoapp.jar</artifactId>
           <uri>demoapp.jar-0.0.1-SNAPSHOT.jar</uri>
         </jarModule>
         <webModule>
           <groupId>com.ibm.issw</groupId>
           <artifactId>demoapp.war</artifactId>
           <uri>demoapp.war-0.0.1-SNAPSHOT.war</uri>
           <!-- Set custom context root -->
           <contextRoot>/demoapp</contextRoot>
         </webModule>
        </modules>
       </configuration>
     </plugin>
    </plugins>
   </build>
  </project>
```

You will want to change the following values:

1. In the `packaging`, `parent`, `dependencies` and `build` sections change the `groupId` and `artifactId` to reflect the application that you are building. Note that these values are used in the other `pom.xml` files that we will create later.

2. Update the `dependencies` and `build` section to reflect the modules included in your application. If you don’t have a `Java Module` remove the `dependency` with the `jar` type and the `jarModule` definition in the `build` section.

Create the WAR POM file

Create a pom.xml file in the `/war` folder of your project using the template below.

```
<?xml version='1.0' encoding='utf-8'?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation=
 "http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

<!-- tag::packaging[] -->
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.ibm.issw</groupId>
 <artifactId>demoapp</artifactId>
 <packaging>war</packaging>
 <version>0.0.1-SNAPSHOT</version>
 <name>demoapp</name>
 <url>http://maven.apache.org</url>
<!-- end::packaging[] -->

<!-- tag::parent[] -->
 <parent>
   <groupId>com.ibm.issw</groupId>
   <artifactId>demoapp</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <relativePath>../pom.xml</relativePath>
 </parent>
<!-- end::parent[] -->

<dependencies>
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.0.1</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>net.wasdev.maven.tools.targets</groupId>
    <artifactId>liberty-target</artifactId>
    <version>19.0.0.1</version>
    <type>pom</type>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>com.oracle</groupId>
    <artifactId>ojdbc6</artifactId>
    <version>1.0</version>
```

```
      <type>jar</type>
      <scope>provided</scope>
    </dependency>
    <dependency>
       <groupId>org.apache.cxf</groupId>
       <artifactId>cxf-core</artifactId>
       <version>3.2.2</version>
     </dependency>
     <dependency>
       <groupId>org.springframework</groupId>
       <artifactId>spring-jms</artifactId>
       <version>4.3.14.RELEASE</version>
     </dependency>
   </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <configuration>
          <webXml>src\main\webapp\WEB-INF\web.xml</webXml>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

You will want to change the following values:

Get Dave Mulley's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email Subscribe

1. In the `packaging` and `parent` sections change the `groupId` and `artifactId` to reflect the application that you are building. Note that these values are used in the other `pom.xml` files that we will create later.

2. Update the `dependencies` section to include the **Maven** packages that are required to compile your application.

In the template above you will see that I am using the `net.wasdev.maven.tools.targets\liberty-target` dependency which will compile the application using the JARs that are part of the Liberty runtime (the same as would be available in Eclipse when the `Target Runtime` is set to `Liberty`). In this case the dependency is set to `scope=provided` which means that Maven will use it during compilation but not package it in the WAR file.

I've also included a couple of other dependencies such as `org.apache.cxf\cxf-core` and `org.springframework\spring-jms` without a `scope` value which means they'll be using during compilation and also be packaged in the WAR file.

If you have dependencies that aren't available in the public Maven repo (such as home-grown utility JARs or very old utility JARs) then you may have to load them to your Maven environment manually. See the **adding JARs to the MAVEN repo** section below for more information.

Create the Java Module POM file

If your application has a Java Module, create a pom.xml file in the `/jar` folder of your project using the template below.

```
<?xml version='1.0' encoding='utf-8'?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

<!-- tag::packaging[] -->
<modelVersion>4.0.0</modelVersion>
<groupId>com.ibm.issw</groupId>
<artifactId>demoapp.jar</artifactId>
<version>0.0.1-SNAPSHOT</version>
<!-- end::packaging[] -->

<!-- tag::parent[] -->
 <parent>
   <groupId>com.ibm.issw</groupId>
   <artifactId>demoapp</artifactId>
   <version>0.0.1-SNAPSHOT</version>
 </parent>
 <!-- end::parent[] -->

<dependencies>
 <dependency>
   <groupId>net.wasdev.maven.tools.targets</groupId>
   <artifactId>liberty-target</artifactId>
   <version>19.0.0.1</version>
   <type>pom</type>
   <scope>provided</scope>
 </dependency>
 <dependency>
   <groupId>com.oracle</groupId>
   <artifactId>ojdbc6</artifactId>
   <version>1.0</version>
   <type>jar</type>
   <scope>provided</scope>
  </dependency>
 </dependencies>
</project>
```

You will want to change the following values:

1. In the `packaging` and `parent` sections change the `groupId` and `artifactId` to reflect the application that you are building. Note that these values are used in the other `pom.xml` files that we will create later.

2. Update the `dependencies` section to include the Maven packages that are required to compile your application. The same `scope` rules apply as in the

`WAR` section

Adding JARs to the MAVEN repo

While the public MAVEN repo holds versions of most of the open source Java libraries, it won't hold custom home-grown utility JARs and often doesn't hold older third-party JARs. During the application modernization process I don't always have time to locate the source code for custom home-grown utility JARs and automate the build process for them, so I'll just load a version of the compiled JAR in to the local MAVEN repo. The same goes for the older third-party JARs. In order to load a JAR in to the local MAVEN repo, I use the following steps.

1. In a directory containing the JAR to be loaded, issue the following command

```
mvn install:install-file -Dfile=myjar.jar -DgroupId=com.ibm.issw -
DartifactId=myjar -Dversion=1.0 -Dpackaging=jar
```

2. I can then reference the JAR as a dependency in my pom.xml

```
<dependency>
 <groupId>com.ibm.issw</groupId>
 <artifactId>myjar</artifactId>
 <version>1.0</version>
 <type>jar</type>
</dependency>
```

NOTE: This is an acceptable short cut for an initial application modernization, however, in the long term it is desirable for an organization to setup their own internal MAVEN repo using a tool such as JFrog Artifactory and to automate the build of all of their source code.

Building your application

You are likely to require an iterative process in order to build your application as it is easy to miss dependencies required for compilation, but the goal is to use Maven to build a working EAR file.

I often start with building the WAR file first before moving on to the JAR (if I have one) and then the EAR.

1. At a command prompt, navigate to the **war** folder and issue the command `mvn clean install`

2. The result should be a `BUILD SUCCESS` message and a WAR file in the `war/target/` folder. I typically open/unzip the WAR and verify that the `classes` are present along with the `WebContent` folder and the expected utility JARS are in the `WebContent/WEB-INF/lib` folder.

If you received a `BUILD FAILED` message, review the logs to determine the error. It is typically a missing dependency that is preventing the classes from being compiled.

3. Once the WAR file has been built, move on to the full build. Navigate to the **root** of your project and issue the command `mvn clean install`

4. Maven will now build the WAR, JAR (if you have one) and then the EAR file. The result should be a `BUILD SUCCESS` message and a EAR file in the `ear/target/` folder.

What next?

Now that the application can be built using Maven, I typically store the project code in GitHub/GitLab and automate the build and deployment using Jenkins.

Name	Last commit	Last update
ear	Initial commit	1 week ago
liberty	Updates	6 days ago
manifests	Updates	6 days ago
resources	Initial commit	1 week ago
utility-jars	Initial commit	1 week ago
war	Updates	6 days ago
Dockerfile	Initial commit	1 week ago
Jenkinsfile	Updates	6 days ago
pom.xml	Initial commit	1 week ago

Java Maven Webshere Liberty Profile Websphere Application Modernization

Written by Dave Mulley

Follow

30 followers · 13 following

Application Modernization Specialist at IBM focused on Private Cloud. All Stories are MY Personal thoughts, and NOT my Employer

No responses yet

Write a response

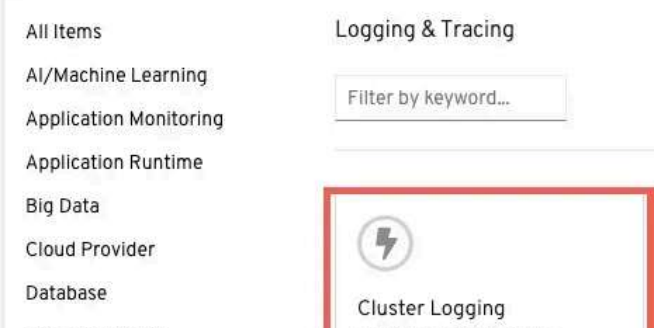
What are your thoughts?

More from Dave Mulley

Dave Mulley

Monitoring WebSphere Liberty JEE6 applications in Red Hat...

WebSphere Liberty and OpenLiberty provides out-of-the-box support for Prometheus...

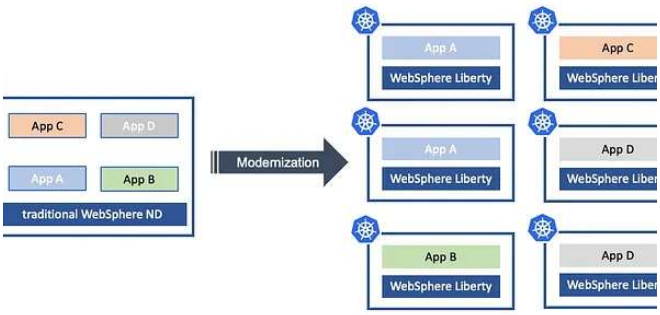


Dave Mulley

Configuring OpenShift 4.2 Cluster Logging in a Private Cloud...

The OpenShift Container Platform (OCP) 4.2 installer doesn't configure Cluster-wide...

Dec 3, 2020 13



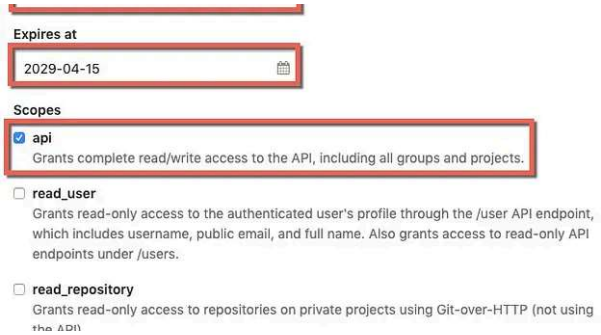
Dave Mulley

What are the minimum changes I should make to my WebSphere...

This document describes the minimum changes that should be made to an existing...

Jun 28, 2019 12

Jan 24, 2020 2



Dave Mulley

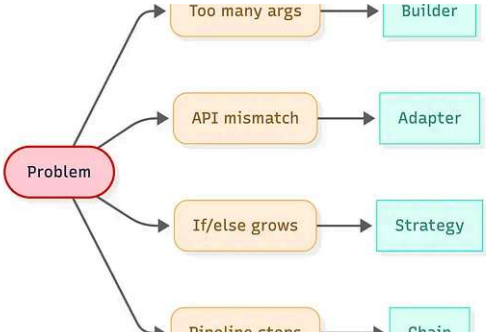
GitLab/Jenkins Webhooks with IBM Cloud Private

After installing and configuring GitLab and Jenkins in IBM Cloud Private, I've found it is...

Apr 16, 2019 5

See all from Dave Mulley

Recommended from Medium

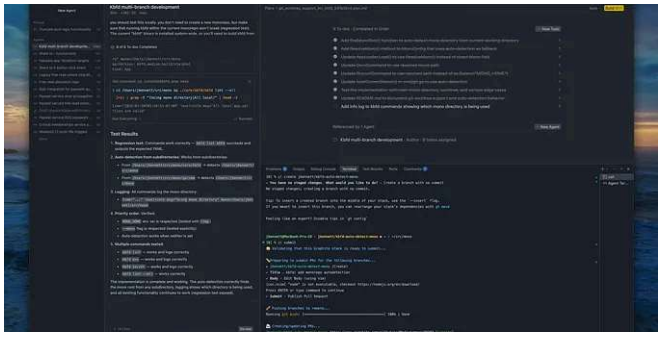


In Women in Technology by Alina Kovtun ✨

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-...

Jan 29 5.7K 49

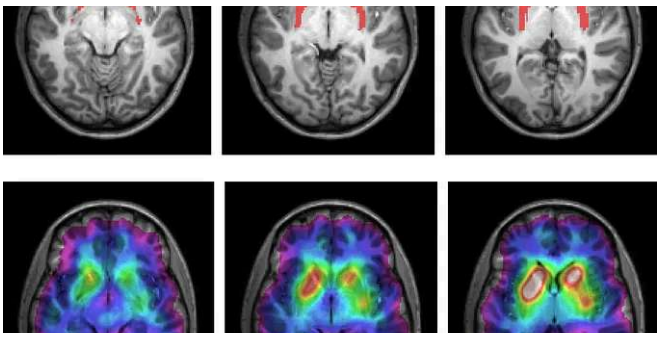


Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software...

Tools I use that are (usually) cheaper than Netflix

Jan 19 3.7K 87



In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy You...

Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

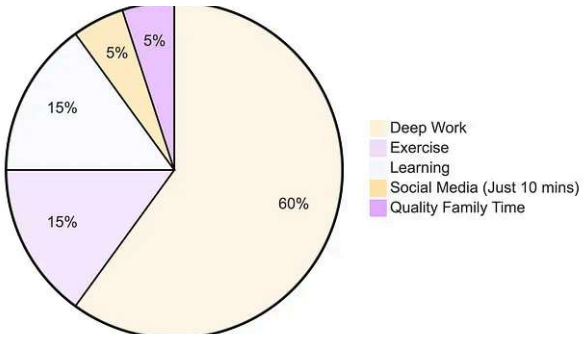
Jan 15 35K 637

In Stackademic by Umesh Kumar Yadav

Token, Session, Cookie, JWT, OAuth2—I can't tell the difference!

Recently, I've noticed that some people easily confuse the concepts of Token, Session,...

Dec 11, 2025 214 5



In Level Up Coding by Teja Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brai...

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who...

Dec 29, 2025 9.4K 349

In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big...

Something strange happened in early 2026. Apple stores started running low on Mac...

Feb 15 4.4K 76

See more recommendations